

Trabajo práctico 1

Análisis de Componentes Principales y Simulación Monte Carlo

1. Introducción

La forma en que se representan los datos es un aspecto fundamental para su análisis y procesamiento en muchas aplicaciones. En particular, cuando los datos presentan una dimensionalidad elevada, trabajar con todas sus variables puede incrementar significativamente el costo computacional y dar lugar a problemas asociados con la llamada **maldición de la dimensionalidad**. Una estrategia para abordar esta problemática consiste en reducir la dimensionalidad de los datos, buscando una representación en un espacio de menor dimensión que conserve la mayor parte de la información relevante, dado que gran parte de la información contenida en el espacio original puede ser redundante. El *Análisis de Componentes Principales* (PCA) es una de las herramientas que permite mitigar este problema, ya que transforma las variables originales en nuevas variables no correlacionadas que concentran progresivamente la mayor parte de la variabilidad de los datos. En este trabajo práctico se explorará el uso de PCA aplicado a los datos de entrada de un clasificador de imágenes. Asimismo, se utilizarán técnicas de simulación Monte Carlo para evaluar la respuesta del sistema frente a perturbaciones en las entradas. El objetivo es brindar un marco práctico para comprender cómo PCA permite concentrar propiedades importantes de los datos en un espacio reducido, facilitando su análisis y visualización. A su vez, se busca comprender la Ley de los Grandes Números mediante la estimación de cantidades de interés empleando simulaciones de Monte Carlo.

1.1. Conceptos básicos sobre Clasificación

La clasificación es el proceso mediante el cual se construye un modelo matemático capaz de predecir a qué categoría pertenece un dato a partir de sus características. Cada categoría representa un conjunto de datos que comparten características comunes. Uno de los enfoques más utilizados es el de clasificación supervisada, que requiere una etapa de entrenamiento en la que el modelo se ajusta a un conjunto de datos con sus clases previamente etiquetadas, Figura 1-(a). Durante esta etapa, el modelo aprende relaciones entre las características de los datos y sus respectivas clases. Una vez entrenado, el modelo puede aplicarse a nuevos datos cuyas clases son desconocidas, con el fin de predecirlas, Figura 1-(b). Para evaluar el clasificador, los datos disponibles se dividen para formar un conjunto de *entrenamiento* y un conjunto para *prueba*, ya que evaluar sobre los mismos datos de entrenamiento no permitiría medir la capacidad de generalización del modelo. Se utilizará como métrica de desempeño el *accuracy* (proporción de aciertos) definido como:

$$\text{accuracy} = \frac{\text{total de aciertos}}{\text{total de datos}} \quad (1)$$

Existen diversos tipos de modelos de clasificación, entre ellos: *logistic regression* (LR) [1], *k-nearest neighbors*, *Naive Bayes*, *neural networks*, *support vector machines*, etc. En este trabajo práctico, utilizaremos *LR* por su simplicidad, sin profundizar en su funcionamiento, ya que no es el objetivo del curso y se utilizará principalmente como herramienta para evaluar cómo influye *PCA* en la representación de los datos. En el apéndice se puede ver un código de ejemplo que muestra el uso de un clasificador *LR*.

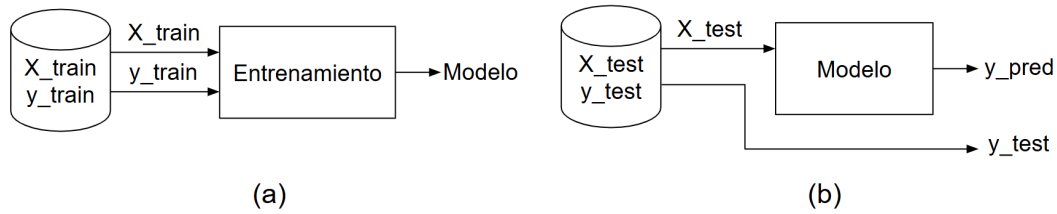


Figura 1: (a) Etapa de entrenamiento; (b) Clasificación usando el modelo entrenado. Cada predicción (y_{pred}) se compara con las etiquetas de prueba y_{test} para calcular el Accuracy.

1.2. Dataset utilizado

Se trabajará con un dataset que contiene 2 clases de imágenes, correspondientes a radiografías de tórax de pacientes *sanos* y con *neumonía*. Estos datos están extraídos de la base de datos gratuita *PneumoniaMNIST* [2]. Para cada clase, se provee un gran número de imágenes de 128×128 píxeles. En la Figura 2 se muestra una de las imágenes a modo de ejemplo. Los datos se dividen en dos directorios, **train** y **test**, que contienen imágenes para entrenamiento (tanto para *PCA* como para el clasificador) y de prueba, respectivamente. Además, en los archivos **train_labels.csv** y **test_labels.csv**, se encuentran las etiquetas correspondientes a cada imagen (siendo **0** $\rightarrow$ **Normal** y **1** $\rightarrow$ **Neumonía**), ya que las imágenes de ambas clases se encuentran mezcladas y solo pueden identificarse mediante estas etiquetas. A continuación se ve la estructura de directorios y archivos:



Figura 2

```

dataset_tp1/
|
+--- train/
|   |
|   +--- PMINST_0001.png
|   +--- PMINST_0002.png
|   +--- ...
+--- test/
|   |
|   +--- PMINST_2429.png
|   +--- PMINST_2430.png
|   +--- ...
+--- train_labels.csv
+--- test_labels.csv

```

2. Ejercicios

Ejercicio 1

- (a) Empleando las imágenes completas (sin aplicar PCA), utilice los datos de entrenamiento para entrenar el clasificador LR . Luego con las imágenes de prueba, aplique la predicción y reporte el *accuracy* resultante.
- (b) Implemente un código para obtener las proyecciones en el espacio de componentes principales, parametrizándolo con el número K de componentes a conservar. Utilice el conjunto de train para obtener la matriz de proyección y el conjunto de test para ser proyectado en el espacio de las dos primeras componentes principales. Luego realice un scatter de dichos vectores (diferencie las clases con colores).
- (c) Para analizar cómo varía el desempeño del conjunto $PCA+LR$ en función de la cantidad de componentes principales, compute el *accuracy* en función de distintos valores de K . Realice un gráfico *accuracy* vs. K , superpuesto al *accuracy* obtenido en el punto (a) (en línea punteada). Analice los resultados obtenidos.

Ejercicio 2

En ocasiones, las imágenes de prueba podrían no tener la orientación adecuada (supongamos que algunas se encuentren rotadas 180° por error), lo que rompería la estructura que mantiene las características semejantes al grupo de pertenencia de dicha imagen. Para estudiar la robustez del conjunto $PCA+LR$ (ya entrenado) frente a este tipo de perturbación, se propone realizar un experimento de *Monte Carlo*. Para ello, cada imagen tomada del conjunto de test durante la simulación deberá ser rotada 180° con probabilidad p , o permanecer sin modificar con probabilidad $1 - p$. Para cuantificar el desempeño frente a esta perturbación, podemos definir la pérdida $L(p)$, que mide cuánto se aleja el *accuracy* A_p (aplicando perturbaciones con probabilidad p), respecto del *accuracy* original A_0 (sin perturbaciones), donde

$$L(p) = A_0 - A_p,$$

En este ejercicio se pide realizar una simulación Monte Carlo (fijando $K = 2$ para PCA) modificando aleatoriamente las imágenes en cada predicción del modelo para producir N_{MC} simulaciones independientes con las que se buscará estimar distintas cantidades de interés. Cada simulación debe estar parametrizada con la probabilidad de perturbación p , para lo cual deben definirse valores dentro del rango $0 < p \leq 0.9$, de acuerdo a la siguientes consignas:

- (a) Para una realización, obtener las dos primeras componentes principales del conjunto de prueba en función de la p utilizada para las perturbaciones. Hacer un scatter de cada caso.
- (b) Estime, mediante el método Monte Carlo, el valor medio del *accuracy* $\mathbb{E}[A_p]$ en función de p . Graficar los resultados.
- (c) Estimar, mediante el método Monte Carlo, la probabilidad de que la pérdida supere cierta tolerancia $\mathbb{P}_p(L(p) > \delta)$. Graficar dicha probabilidad estimada en función de p considerando una tolerancia de $\delta = 0.1$.
- (d) Para cada valor de p , represente mediante un histograma la distribución de los *accuracies* obtenidos en las simulaciones Monte Carlo realizadas.

Aclaración: El modelo *PCA+LG* debe entrenarse una única vez utilizando el conjunto de entrenamiento **sin perturbar**. Las rotaciones aleatorias se aplican exclusivamente a las imágenes del conjunto de test durante las simulaciones de Monte Carlo.

Ejercicio 3

Justifique mediante la *Ley de los Grandes Números*, que el estimador de la probabilidad pedida en el Ejercicio 2-(c), se resuelve como el promedio de las funciones indicadoras del evento de interés:

$$\hat{\mathbb{P}}_p(L(p) > \delta) = \frac{1}{N_{MC}} \sum_{i=1}^{N_{MC}} \mathbb{1}\{L_i(p) > \delta\}$$

3. Conclusiones

Como conclusiones, elabore un resumen breve y conciso comentando características que considere relevantes del método propuesto en este trabajo y los resultados obtenidos, así como dificultades encontradas y cómo fueron abordadas.

4. Apéndice

En el siguiente código se muestra un ejemplo de train y test del modelo *LR* usando el módulo `LogisticRegression` de `sklearn`:

```
# Módulos necesarios para clasificación con Regresión Logística
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression

# Definición del modelo de clasificación de Regesión Logística
lr_model = LogisticRegression(max_iter=2000)

# Entrenamiento del modelo
lr_model.fit(X_train, y_train)

# Validación del modelo
y_pred = lr_model.predict(X_test)
acc = accuracy_score(y_test, y_pred)
```

Aclaraciones:

- `X_train`: matriz cuyas filas son las realizaciones del conjunto de entrenamiento.
- `y_train`: vector con las etiquetas de cada clase asociada a cada vector en `X_train`.
- `X_test`: matriz cuyas filas son las realizaciones de los vectores del conjunto de prueba.
- `y_test`: vector con las etiquetas de cada clase asociada a cada vector en `X_test`.
- `y_pred`: vector con los resultados de la calificación de cada vector de prueba.
- `acc` (accuracy): es la proporción de predicciones correctas comparando `y_test` e `y_pred`.

A continuación se muestran operaciones básicas con imágenes usando el módulo PIL (Python Imaging Library):

```
# Módulo para manejo de imágenes
from PIL import Image

img = Image.open(img_path).convert("L") # abrir una imagen en grises
img_matrix = np.array(img)             # convertir imagen a numpy
img_rotada = np.rot90(img_matrix, 2)   # imagen rotada 180° (2 * 90°)
```

5. Condiciones de entrega

- **Archivo ZIP:** en un archivo comprimido en formato ZIP, de nombre **TP1_GXX.zip** (donde XX es el número de grupo, ej: **TP1_G01.zip**) debe incluirse tanto el informe como el código fuente del trabajo. Este archivo debe subirse al recurso disponible en el campus.
- **Informe:** debe ser en formato PDF (**no se aceptarán otros formatos**) y con nombre: **TP1_GXX.pdf**. Es condición necesaria para la aprobación cumplir con las [pautas](#) para la presentación de informes estipuladas en el campus.
- **Código:** el código debe ser en un archivo de tipo cuaderno (**ipynb**). Los Ejercicios 1 y 2 pueden entregarse en cuadernos separados para mayor comodidad.
Importante: “No debe incluirse la carpeta de imágenes entre los archivos entregados”.
- Se recuerda a los estudiantes que las entregas deben ser un producto original de cada grupo, por lo que se les pide revisar la sección 6 del programa de la materia y el Código de Honor y Ética de la Universidad.

Referencias

- [1] Mahmood, B. (2016). *How to Predict Yes/No Outcomes Using Logistic Regression*. Medium. Disponible en: [Enlace](#).
- [2] Yang et al. (2024). MedMNIST+, PneumoniaMNIST (128×128). Disponible en: [Enlace](#).